

Faculty of Engineering & Applied Science

Mobile Application Development SOFE 4640U

Novemeber 8th, 2026

Student ID	Full Name
100827346	Sami Salah

Video Link:

1. Overview

This GTA Finder application was developed following a modern android architecture to provide dynamic location management and geospatial mapping in the Greater Toronto Area. The architecture uses a data-access layer pattern with a folder for the data related files and three additional files pertaining to the UI. This application uses a local SQLite database for offline data storage and integrates Google Maps Android SDK v18 for rendering the map with accuracy. A lot of the structure and implementation of the database and utilization came from lab 3 and lab 4. We were able to use a similar structure in the creation of this application.

2. Database Implementation

As previously mentioned all the data files are located in a specified data folder. We have a GtaData.kt which is the object file which contains a list of 100 locations in the GTA. This is part of the initial setup which populated the database and the application when first starting when the database was empty. Our MainActivity.kt checks if the database is unpopulated and uses GtaData's dataset to insert all 100 locations into the location table using LocationDao.insertLocation().

Included is a location.kt data class file which contains the layout of each location in the application. This file implements the 5-column requirement with auto-generation constraints enforced at the database level:

- id (INTEGER PRIMARY KEY AUTOINCREMENT): Unique record identifier generated automatically upon row insertion.
- address (TEXT NOT NULL): String description of the GTA location (e.g., Oshawa, Scarborough, Downtown Toronto).
- latitude (REAL NOT NULL): Double-precision floating-point value specifying the north-south geographic coordinate.
- longitude (REAL NOT NULL): Double-precision floating-point value specifying the east-west geographic coordinate.
- created_datetime (DATETIME DEFAULT CURRENT_TIMESTAMP): Timestamp auto-populated at creation.

We also maintain persistence using Android's SQLiteOpenHelper framework using two classes, LocationDbHelper from the LocationDbHelper file and LocationDao from the LocationDao file. This ensures that all SQL queries remain encapsulated inside the data access object(dao) layer and are isolated from the main activity lifecycle.

3. Google Maps SDK Integration

Mapping the geospatial location is done using SupportMapFragment along with the OnMapReadyCallback class that is defined in MainActivity. Whenever onMapReady(googleMap: GoogleMap) runs, the camera for the map focuses on the Greater Toronto Area (43.6532, -79.3832). As the query runs, markers are positioned on the map using MarkerOptions() with title being location address and snippet coordinates.

4. Queries and Filtering

The query method connects the user's inputs to SQLite searches. SearchView.OnQueryTextListener listens for the user's input live in MainActivity. Upon input of any text, the following SQL query with parameterized strings is run using LocationDao.queryByAddress(query):

```
SELECT * FROM location WHERE address LIKE '%' || ? || '%' ORDER BY id DESC
```

The cursor returned gets mapped to a list of LocationItem data objects. The list is used to update the RecyclerView and also updates the map markers with focus on the main match.

5. Add, Update and Delete CRUD Operations

The entry management of location is performed using the ManageLocationActivity file which handles both insertion and modification as shown below:

- **Add Mode**: Starts after pressing the "+ Add New Location" button. Inputs include Address, Latitude, and Longitude provided by the user. Validation checks for non-empty strings as well as the correct range of coordinates, (-90 to 90 for latitude and -180 to 180 for longitude). Clicking submit calls the LocationDao.insertLocation() method.
- **Update Mode**: Starts selecting the element from the RecyclerView list in MainActivity. Intent extras carry the id, address, latitude, and longitude of the selected row and prefill the input fields. Clicking on "Update Location" calls LocationDao.updateLocation().
- **Delete Mode**: In Update Mode, there is a special button "Delete Location". Clicking it triggers LocationDao.deleteLocation(id).

After completion of ManageLocationActivity, the function finish() is called and control is returned to MainActivity, which upon its onResume() method re-queries the database and refreshes both markers and list elements.

AI Work Disclosure

Generative AI tools (ChatGPT/Gemini) were consulted for technical guidance regarding SQLite database helper initialization, syntax structure for Google Maps SDK camera bounds, and constructing the initial seed array of 100 GTA geographic coordinates. There were initial set backs with spacing components and utilizing LocationDao methods in MainActivty that were solved using AI. All other activity components, UI layouts (XML), input validation logic, data access objects, intent parameters and finally the report were written, refined, integrated, and verified manually by me.